R Creative Factory

A self-improving creative engine for growth marketing. Generates ad creatives, runs structured experiments, monitors performance, allocates spend mathematically, and compounds learnings across every cycle.

How It Works

Creative Factory is a closed-loop system. It generates creatives informed by what worked before, tests them with statistical rigor, and feeds results back into a persistent knowledge layer. Every cycle makes the next cycle smarter.

1. **Load Positioning** — What to say. Living document with messaging by segment, channel, and geo.
2. **Read Knowledge Layer** — How to say it. Winning patterns, losing patterns, fatigue signals from past cycles.
3. **Generate Creatives** — Multi-variant ads per channel × geo × segment. Each cites the insights that informed it.
4. **Run Experiments** — Structured A/B tests with hypotheses, confidence thresholds, and sample size plans.
5. **Monitor + Allocate** — Detect creative fatigue, CAC shifts, payback drift. Reallocate spend via Thompson Sampling.
6. **Learn** — Extract insights from experiments, update creative DNA, generate new hypotheses.

↑ Loop closes — knowledge layer grows — next generation is smarter ↑

Commands

COMMAND	WHAT IT DOES
`/creative-factory status`	Dashboard — insights count, active experiments, alerts, budget, compound growth
`/creative-factory generate [channel]`	Generate creative variants for paid_search, paid_social, aso, or lifecycle
`/creative-factory monitor`	Run performance alerts — creative fatigue, CAC shifts, payback drift
`/creative-factory allocate`	Thompson Sampling budget reallocation across experiment arms

`/creative-factory learn`	Close experiments, extract insights, update knowledge layer, generate hypotheses
`/creative-factory ingest`	Load performance data — paste, CSV, or simulated
`/creative-factory report`	Daily performance digest with action items

What Each Part Does

POSITIONING DOCUMENT

A living artifact — not a static brief. Contains messaging by segment (developer, non-technical builder, enterprise, student), by channel (search, social, ASO, lifecycle), and by geo (US, India, EU, LATAM). A *performance signals* section updates automatically when the learning loop discovers that certain messages work better than expected.

KNOWLEDGE LAYER

The core of the compound effect. Three files work together:

- **insights.json** — Every validated finding, with confidence level, sample size, and channel/geo/segment tags. Append-only — history is the asset.
- **creative-dna.json** — Winning and losing patterns organized by channel. What hooks work, what CTAs convert, what formats fatigue fastest.
- **fatigue-signals.json** — How long each creative type lasts before decay, by channel and geo. Drives proactive refresh scheduling.

CREATIVE GENERATION

Not blind prompting. The engine reads the full knowledge layer before generating anything, then produces variants that apply proven patterns (80%) with controlled exploration (20%). Every variant cites which insights informed it by ID — that's the audit trail that makes compounding visible.

Each variant is routed to a specific platform:

Meta (IG / FB)
Non-technical builders, students. Visual-first, UGC style, carousels.

LinkedIn
Enterprise teams, B2B developers. Outcome framing, security qualifiers.

Twitter / X
Developers, builder community. Capability demos, "look what I built."

YouTube Shorts / TikTok
Students, India/LATAM. Short-form video, mobile-first.

Visual mockups can be rendered per variant using a codified design system with brand tokens, layout patterns, and platform-specific canvas sizes.

EXPERIMENT TRACKING

Every test starts with a hypothesis that traces back to existing insights. The schema includes the statistical plan (confidence threshold, required sample size), the variants, and multi-stage conversion tracking (signup → first app → deploy → paid). This catches signal quality issues early — high signup rate but low activation means the creative is attracting the wrong people.

PERFORMANCE MONITORING

Four automated alert types:

- **Creative fatigue** — CTR declines >20% over 14 days
- **CAC shift** — CPA increases >30% over 7 days
- **Payback drift** — Days-to-payback increases >25% over 14 days
- **Conversion quality** — Signup-to-paid rate drops >15% over 7 days

Each alert includes a recommended action sourced from the knowledge layer.

BUDGET ALLOCATION (THOMPSON SAMPLING)

Not gut calls. Each creative variant is modeled as a Beta distribution. The system samples from each arm's posterior, allocates budget proportional to probability of being best, and maintains an exploration floor (15%) to keep learning from underexplored variants. When confidence exceeds 95%, it recommends declaring a winner and retiring losers.

LEARNING LOOP

This is where the system compounds. When an experiment reaches confidence threshold, the loop:

- Extracts 1-3 specific insights with confidence and sample size
- Updates creative-dna.json with new winning/losing patterns
- Checks for meta-patterns (when 3+ insights point the same direction)
- Generates new hypotheses from untested insight combinations
- Updates the positioning document's performance signals

> **The compound effect:** Cycle 1 starts with zero insights. By cycle 3, the system has 22 insights and 4 meta-patterns. Every generation after that is informed by everything the system has ever learned. The knowledge layer is append-only — it can only get smarter.

File Structure

```
~/.claude/data/creative-factory/
├── config.json                      # Brand config, channels, geos, budgets
├── positioning/
│   ├── active.json                  # Machine-readable positioning (generation input)
│   └── replit-positioning.md        # Full positioning document
├── knowledge/
│   ├── insights.json                # Compound learning store (append-only)
│   ├── creative-dna.json            # Winning/losing patterns by channel×geo×segment
│   └── fatigue-signals.json         # Decay signatures and refresh schedules
├── experiments/
│   ├── active.json                  # Running A/B tests
│   ├── completed.json               # Archived experiments with results
│   └── hypotheses.json              # Auto-generated from insight combinations
├── performance/
│   ├── metrics.json                 # Performance data
│   ├── alerts.json                  # Active alerts (fatigue, CAC, payback)
│   └── allocation.json              # Thompson Sampling state
├── scripts/
│   ├── thompson.py                  # Multi-armed bandit allocation engine
│   └── simulate_performance.py      # Realistic mock data generator
├── creatives/
│   ├── DESIGN-SYSTEM.md             # Brand tokens, layout patterns, specs
│   └── *.png                        # Rendered mockups and brand assets
```

```
└── reports/
    └── YYYY-MM-DD.md                # Daily performance digests
```